

STEMMA: SONG-TO-STEM MULTI-AUDIO REASONING FOR LARGE AUDIO LANGUAGE MODELS

Hoyeol Sohn¹ , Wonil Kim² , Keunhyoung Kim² , Sangeun Kum² , Taehyoung Kim² ,
Dongjoo Moon² , Theerasak Charoenchob² , Teeratep Weerapang² , Jongpil Lee² , Juhan Nam¹ 

¹ Graduate School of Culture Technology, KAIST, Daejeon, Republic of Korea

² Neutune, Seoul, Republic of Korea

ABSTRACT

Music understanding often requires comparing excerpts and reasoning about relationships among songs, sections, and stems. However, existing large audio-language models (LALMs) and music question-answering datasets typically operate on single recordings or compare independently sampled tracks with no known production relationship. We introduce STEMMA, a multi-audio music question-answering framework built around production provenance: whether excerpts originate from the same track or section, and which stems belong to which mixtures. Because such relations are sparse under conventional audio-first sampling, STEMMA adopts a relation-first construction strategy: it first specifies a target relation and then queries the catalog for excerpts that satisfy it and hard negatives that do not. Labels are determined directly from catalog provenance rather than generated by a language model from metadata. We build STEMMA-Bench for evaluation and a track-disjoint training set, STEMMA-Instruct. Fine-tuning two LALMs on STEMMA-Instruct improves multi-audio reasoning, with the largest gains on structural relations directly determined by the catalog, while preserving single-audio music understanding.

Index Terms— multi-audio reasoning, music understanding, audio-language models, instruction tuning, hierarchical music structure

1. INTRODUCTION

A line of music-specialized large audio-language models (LALMs) has shown that instruction tuning on music-specific supervision improves captioning, question answering, and analysis of music theory, structure, and instrumentation [1, 2]. Nevertheless, most models and training corpora still treat each recording as an independent item.

Emerging generative music experiences increasingly require models to reason about relationships among songs, sections, stems, and musical attributes [3]. Retrieval, arrangement, and sample discovery therefore require a model to determine whether excerpts originate from the same track or section [4] and to compare tempo, key, instrumentation, harmony, genre, and mood [5, 6]. This setting requires *cross-audio grounding*: evidence must be localized in each input before a relation across inputs can be inferred. Conversational music agents orchestrate MIR and generation tools from natural language but typically rely on intermediate tool outputs and stored musical attributes rather than jointly reasoning over multiple audio inputs [7]. As a result, relations requiring direct comparison between signals remain outside their native reasoning scope.

Recent work has begun to address this limitation: explicit multi-audio instruction tuning improves comparison and reasoning [8, 9], while comparative question answering has been extended to pairs of independently collected music tracks [10] and to natural-language explanations of audio difference [11]. Performance nevertheless degrades as the candidate set grows [12]. However, prior work does not systematically exploit the known song–section–stem provenance available in music production data.

We propose STEMMA, a multi-audio music question-answering framework whose answers are determined by known song–section–stem provenance: whether two excerpts share a track or a section, and which stem belongs to which mixture. Because such relations are too sparse under conventional audio-first sampling, STEMMA adopts a *relation-first* strategy: the target relation is specified before audio is sampled, and a single catalog query retrieves both matching examples and near-misses that violate it. Our contributions are: (1) STEMMA-Bench, a held-out benchmark over sections and section stems drawn from full songs rather than over independently sampled tracks, and (2) STEMMA-Instruct, a track-disjoint training set on which we instruction-tune two LALMs, raising aggregate multi-audio accuracy while single-audio music understanding is retained. Project page: <https://github.com/hoys048/STEMMA>.

2. RELATED WORK

Music understanding in LALMs. LLark unifies captioning, understanding, and reasoning over open-source music annotations in a single instruction-tuning format [1]. Music Flamingo scales this approach with synthetic supervision covering harmony, structure, timbre, lyrics, and cultural context [2], while MOSS-Music is an open music audio-language model trained with a similar pipeline [13]. These models are trained and evaluated one recording at a time.

Multi-audio grounding in LALMs. Beyond Single-Audio constructs paired-audio tasks and shows that targeted fine-tuning can add two-audio capability without substantially sacrificing single-audio performance [8]. PolyAudio extends this to interleaved contexts and four skill families, including comparison and temporal reasoning [9]. MUGEN evaluates grounding among multiple audio candidates, finding that performance falls with more inputs and that acoustic comparison is the main bottleneck [12].

Comparative music QA. Jamendo-MT-QA samples full-track pairs and uses an LLM to generate comparative questions from per-track captions, metadata, and single-track QA, followed by human- and LLM-based filtering [10]. Independently sampled tracks share no provenance, so the questions can only compare attributes. STEMMA instead draws sections and stems from the same song, enabling structural questions that independent pairs cannot express.

This work was conducted while the first author was an intern at Neutune.

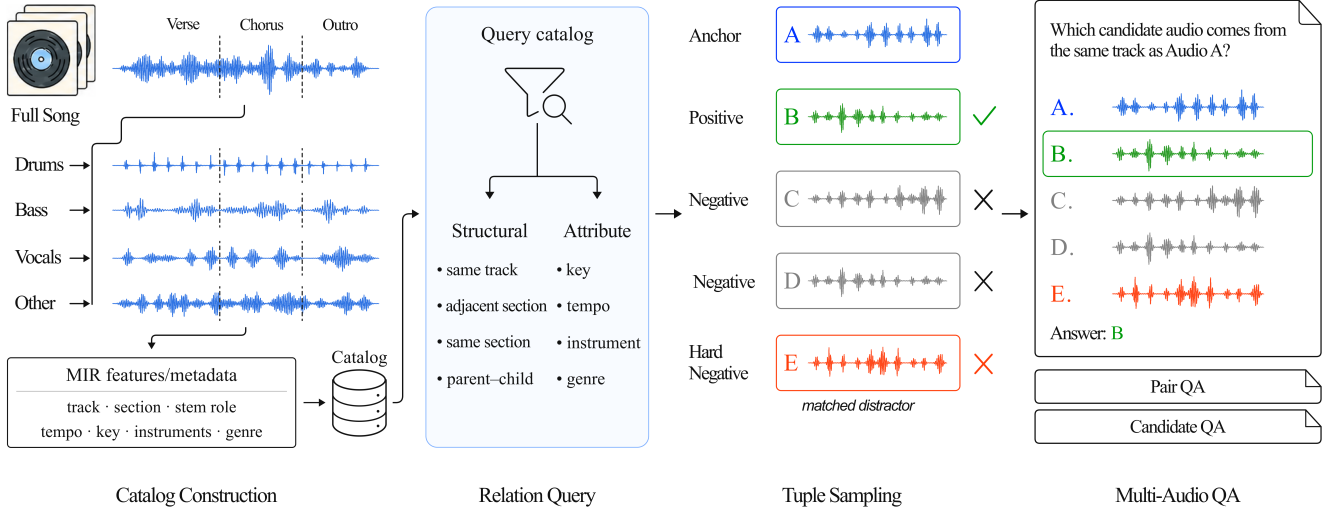


Fig. 1. Overview of the proposed relation-first generation pipeline. Hierarchical music units are indexed by structural and musical-attribute relations. A relation is stated over catalog fields, and the tuples that satisfy or violate it are retrieved by query before being rendered as multi-audio instructions.

3. RELATION-FIRST MULTI-AUDIO DATA

3.1. Why the relation comes first

A relation-labeled tuple can either be sampled and retained when the relation happens to hold or queried directly. Structural relations are sparse. Indeed, two excerpts drawn independently from N tracks share a track with probability $O(1/N)$, and sharing a section is rarer still, so sampling wastes most draws on rejected tuples, whereas direct querying eliminates this rejection cost by retrieving tuples that satisfy the relation.

Specifying the relation first also controls how similar the negative examples are to the correct one. This is defined by explicit conditions rather than by chance sampling. Because the answer is determined from catalog fields before any text is generated, neither the label nor the difficulty of the example depends on language-model output. A new task therefore only requires a new task definition and the query used to retrieve its examples, while reusing the same catalog and data format.

3.2. Building the catalog

The first step is to turn music hierarchy into a queryable table (Fig. 1). Each track is divided into ordered sections with section labels, and each section is separated into individual musical roles. A single recording therefore yields many excerpts with known relationships. Every excerpt corresponds to one section and contains either the full section mixture or one separated role. We call these a *section* and a *section stem*, respectively. Each row stores four provenance fields: track, section index, section category, and stem role. It also stores musical attributes used by the relation queries, including tempo, key, instruments, and genres. We use production metadata when available; otherwise, these fields can be estimated from full-song audio using standard MIR tools for structure segmentation, source separation, beat tracking, key estimation, and instrument tagging. This allows other full-song corpora to be represented in the same schema. The catalog uses a relational schema, with one row per excerpt in the main table and separate tables for set-valued genre and instrument fields.

3.3. Relation queries

A *relation* is defined as a declarative record. It specifies the source of the anchor, correct candidate, and distractor, as well as the conditions for valid positive and hard-negative tuples. For example, a *parent-child* relation pairs a section with one of its stems. Each condition applies one of eight operators to a catalog field: *same* and *different*; *close* and *far* with a tolerance; *overlap* and *no_overlap* with a minimum; and *greater* and *less*. Table 1 lists the positive conditions and corresponding questions for each relation. For each relation, hard-negative conditions preserve selected context while violating the target relation. All excerpts in a tuple are distinct and non-silent.

A *QA format* specifies how the audio evidence is presented and how the answer is expressed. In *pair QA*, a question about two excerpts is answered with yes/no or a short value. In *candidate QA*, an anchor is paired with four audio candidates and the model selects one, forming a multiple-choice question whose options are audio rather than text. Because relation and format are independent, one relation can define multiple tasks: *same track*, for example, can ask either “Do these two excerpts come from the same track?” or “Which candidate comes from the same track as the anchor?”

A single SQL query over the catalog generates each task tuple, its answer, and the answer position. The query has one branch per role, while a hash determines the slot of the correct candidate. One *same section* instance, with identifiers written as *track-section-role*, is:

catalog (samples)			
anchor	726150-Verse-vocals	hard-negative	726150-Chorus-drums
positive	726150-Verse-drums	negative	128662-Verse-drums
			1074236-Inst-drums
relation same section		compiled query	
positive	same(track)	b.track.id = a.track.id	
	same(section)	b.section.id = a.section.id	
	different(role)	b.stem.role <> a.stem.role	
hard-negative	same(track)	b.track.id = a.track.id	
	different(section)	b.section.id <> a.section.id	

The positive example shares the anchor’s section but uses a different role. The hard negative keeps the same track but comes from a different section, making it another stem from elsewhere in the

Table 1. The nine relations: the conditions a positive tuple must satisfy, the units each is instantiated over, and the question a candidate QA instance asks. A pair QA instance asks the same relation of two excerpts. In the Units column, *Sect.* denotes a section excerpt, and *Stem* denotes a section stem.

Relation	Positive conditions	Units	Question
<i>Structural</i>			
same track	<code>same(track), different(section)</code>	Sect., Stem	Which candidate audio comes from the same track as Audio A?
adjacent section	<code>same(track), section+1</code>	Sect., Stem	Which candidate excerpt immediately follows Audio A in the original track?
same section	<code>same(track), same(section), different(role)</code>	Stem	Which candidate audio is a different isolated component from the same recorded section as Audio A?
parent-child	<code>same(track), same(section)</code>	Sect. → Stem	Which candidate audio is contained within Audio A?
<i>Attribute</i>			
same key	<code>same(key), different(track)</code>	Sect., Stem	Which candidate audio shares the same musical key as Audio A?
same tempo	<code>close(tempo,3), different(track)</code>	Sect., Stem	Which candidate audio has a similar tempo to Audio A?
tempo order	<code>greater(tempo,3), different(track)</code>	Sect., Stem	Which candidate audio is faster in tempo than Audio A?
instrument overlap	<code>overlap(instr,2), different(track)</code>	Sect.	Which candidate audio shares at least two instrument tags with Audio A?
genre overlap	<code>overlap(genre,2), different(track)</code>	Sect.	Which candidate audio shares at least two genre tags with Audio A?

same song. A set-valued condition such as `overlap(genre,2)` is implemented as a join over the corresponding side table with a minimum-count threshold.

4. EXPERIMENTAL SETUP

Corpora. We build the catalog from three full-song collections. An in-house production set of 15,856 songs from a licensing-cleared catalog carries authored metadata: section boundaries, separated stems, tempo, key, genre, and instrumentation. The other two are public and carry no such annotation, and are included to show that the catalog does not depend on production data: 69,297 songs from JamendoMaxCaps [14], and 12,119 songs under 600 seconds from a commercially reusable subset of the Free Music Archive [15], restricted to public-domain, CC-BY, and CC-BY-SA licenses at a pinned release. Before generation, we remove thirteen JamendoMaxCaps tracks whose audio matches evaluation items; the corpus counts above are after this exclusion. A full song is represented by its sections concatenated in order, or by the original recording where no segmentation exists.

Catalog construction. For the two public corpora every field of Section 3.2 is estimated from audio: sections from SongFormer [16], stems from SCNet [17], key and tempo from Essentia [18], and genre and instrument tags from Discogs-EffNet models [19]. Estimated fields are stored exactly like authored ones, so a relation is written once and holds over every corpus without change.

Training data. STEMMA-Instruct is the instruction-tuning half of STEMMA. Its two roles, Relation MultiQA and coreference, contribute 97,272 instances each, so STEMMA-Instruct holds 194,544 instances with one to five audio inputs each. Relation instances are 48% section-only, 45% section-stem-only, and 7% mixed. Relation MultiQA is the multi-audio data of Section 3, generated at one instance per full song. Coreference is assembled from a pool of single-audio full-song QA, built by adapting Music Flamingo’s synthetic-caption recipe [2] and the MOSS-Music-Data-Pipeline¹: an audio-language model drafts an audio-grounded caption and a text LLM refines it against source metadata and MIR estimates.

Coreference. A multi-audio instruction carries references that mean nothing outside its context, such as Audio A, so answering it requires grounding language to the correct input index [9]. Two to four full songs from distinct tracks are concatenated under separate labels, and the target answer repeats each song’s own full-song answer under its matching label, so a correct answer places each description under the label of the audio it came from.

Models and instruction format. We instruction-tune two backbones, Music Flamingo and MOSS-Music [13]. Both are already instruction-tuned for single-audio music understanding over full songs, the setting STEMMA-Instruct extends rather than replaces. Each is fully fine-tuned for one epoch with batch size 128 and learning rate 1×10^{-5} on two B200 GPUs. Every instance places one `<audio>` placeholder per input on its own labeled line and uses letters rather than indices. Pair QA uses Audio A and Audio B; candidate QA uses Audio A with Candidate B–E. Each prompt closes with an explicit response-format instruction. Training and evaluation use the same contract, re-serialized into each backbone’s native audio interface.

Evaluation. Multi-audio evaluation uses STEMMA-Bench, our held-out benchmark of candidate QA posed over sections and section stems drawn from full songs. It is generated by the same procedure as STEMMA-Instruct’s Relation MultiQA but from MUSDB18 [20], MedleyDB [21], and MoisesDB [22], thus the two share no corpus and its stems are authored rather than separated. It holds 300 questions (100 per corpus) over 481 songs, of which 127 test structural and 173 musical-attribute relations. Every item was checked by hand, and those judged musically implausible were replaced under the same source, relation and answer strata until all 300 were accepted. Two further multi-audio benchmarks are reported beside it: MUGEN: Music Analysis [12], and 2,010 short-answer items from Jamendo-MT-QA [10]. Single-audio music understanding is measured on MMAU-Pro: Music [23], MMAU: Mini-Music [24], MMAR: Music [25] and RUL-MuChoMusic [26], the last under its four-fold protocol. Direct systems answer with greedy decoding, and each component of a cascade uses its own default sampling parameters; a response that cannot be parsed into the requested format counts as incorrect.

5. RESULTS

5.1. System-level comparison

Table 2 compares direct audio-QA models with cascaded systems, in which an audio-language model captions every input independently and Qwen3.6-35B-A3B [27] reasons over the resulting captions.

Tuning on STEMMA-Instruct improves every multi-audio benchmark for both backbones. Music Flamingo gains 28.8 points on MUGEN: Music Analysis, 6.4 on Jamendo-MT-QA, 59.8 on STEMMA-Bench’s structural relations and 17.9 on its attribute relations; MOSS-Music gains 0.8, 2.5, 23.6 and 0.6. The gain is larger for backbones with lower pretrained performance: MOSS-Music is already the strongest pretrained direct system on MUGEN: Music

¹github.com/wx9Songs/MOSS-Music-Data-Pipeline.

Table 2. System-level accuracy across multi-audio reasoning and single-audio music understanding. Each cascade supplies independently generated audio captions to Qwen3.6-35B-A3B. Best and second-best scores are bold and underlined, respectively. Parse failures count as incorrect.

System	Multi-Audio QA				Single-Audio QA			
	MUGEN	Jamendo-MT-QA	STEMMA-Bench		MMAU-Pro	MMAU	MMAR	RUL-MuChoMusic
	Music Analysis		Struct.	Attr.	Music	Mini-Music	Music	
<i>Cascaded systems</i>								
Audio Flamingo 3 [28] → Qwen3.6	0.5000	0.7169	0.4803	0.2428	0.5896	0.7006	0.4138	0.6828
Music Flamingo → Qwen3.6	0.5400	0.8716	0.4803	0.2428	0.5367	0.6677	0.4039	0.6114
MOSS-Audio [29] → Qwen3.6	0.5320	0.7741	0.4094	0.2254	0.4915	0.5689	0.4236	0.5078
MOSS-Music → Qwen3.6	0.5800	<u>0.8692</u>	0.5039	0.2601	0.4972	0.5778	0.3892	0.5082
<i>Direct QA systems, pretrained</i>								
Qwen2-Audio [30]	0.1960	0.5348	0.2283	0.2254	0.4337	0.6018	0.3941	0.4804
Qwen2.5-Omni [31]	0.5400	0.7582	0.5354	<u>0.2948</u>	0.6601	0.6826	0.5123	0.4530
Audio Flamingo 3	0.1720	0.5627	0.2126	0.1676	0.6361	0.7455	0.4828	<u>0.6590</u>
MOSS-Audio	0.3720	0.6433	0.3780	0.2370	0.6481	0.7635	0.5517	0.5179
Music Flamingo	0.2760	0.7836	0.2205	0.1734	0.6700	0.7395	0.4828	0.2207
MOSS-Music	<u>0.6240</u>	0.8373	0.3543	0.2832	<u>0.7193</u>	<u>0.7665</u>	<u>0.5764</u>	0.5792
<i>Direct QA systems, fine-tuned</i>								
Music Flamingo w/ STEMMA-Instruct	0.5640	0.8478	0.8189	0.3526	0.6770	0.7455	0.5123	0.3012
MOSS-Music w/ STEMMA-Instruct	0.6320	0.8622	<u>0.5906</u>	0.2890	0.7228	0.7844	0.6010	0.5866

Table 3. Ablation over the two multi-audio roles of STEMMA-Instruct; Relation MultiQA + Coreference is STEMMA-Instruct itself. Each average weights the columns of Table 2 equally within its group. Parentheses report percentage-point changes from the backbone’s pretrained model.

Training data	Multi-Audio Avg.	Single-Audio Avg.
<i>Music Flamingo</i>		
Pretrained	0.3634	0.5282
Coreference only	0.4189 (+5.55)	0.5345 (+0.63)
Relation MultiQA only	0.5463 (+18.29)	0.5538 (+2.56)
Relation MultiQA + Coreference	0.6458 (+28.24)	0.5590 (+3.08)
<i>MOSS-Music</i>		
Pretrained	0.5247	0.6603
Coreference only	0.5331 (+0.84)	0.6772 (+1.69)
Relation MultiQA only	0.5597 (+3.50)	0.6640 (+0.37)
Relation MultiQA + Coreference	0.5935 (+6.88)	0.6737 (+1.34)

Analysis (0.6240) and moves least there, while Music Flamingo starts at 0.2760 and gains most. Single-audio music understanding is also retained for both backbones and improves on every benchmark, with the largest gain coming from Music Flamingo on RUL-MuChoMusic (+8.1).

What separates the benchmarks is how far a cascade gets. On Jamendo-MT-QA the best system in the table is a cascade, 0.8716 against 0.8622 for the best tuned model, which matches Jamendo-MT-QA’s own report that caption-based baselines remain competitive with multi-audio systems [10]. On STEMMA-Bench’s structural relations the ordering reverses and the margin is wide: the best tuned model reaches 0.8189 against 0.5039 for the best cascade, and tuned Music Flamingo exceeds its own cascade by 33.9 points. Comparisons between independently collected tracks are largely answerable from per-track descriptions; relations among excerpts that share a source are not.

Among pretrained systems the strongest on STEMMA-Bench is Qwen2.5-Omni, which is not music-specialized: 0.5354 on the structural relations against 0.3543 for MOSS-Music and 0.2205 for Music Flamingo, and 0.2948 on the attribute relations, second in that column only to the tuned Music Flamingo. These results suggest that native support for multiple audio inputs may matter more here than music-specific pretraining.

5.2. Relation-level analysis

Table 2 splits STEMMA-Bench into the relations whose answer the catalog records as provenance and those whose fields are estimated by MIR components, and the two groups behave differently. Tuning moves the structural relations by 59.8 points for Music Flamingo and 23.6 for MOSS-Music, and Music Flamingo reaches 0.8189, the highest score in that column. The attribute relations top out at 0.3526 against a chance of 0.25, and most systems in the table sit at or below chance on them.

The cascades give the same reading from the other side. With a strong captioner and a text model they reach 0.4094–0.5039 on the structural relations but only 0.2254–0.2601 on the attribute relations. Why the attribute relations stay hard is open. They depend on intentionally small differences, e.g., three beats per minute, one key label, or two shared tags. Such fine-grained differences may be lost both in captions and in the MIR estimates used to populate the catalog.

5.3. Training-mixture analysis

Table 3 separates the two multi-audio roles of STEMMA-Instruct. Coreference alone moves multi-audio accuracy less, +5.55 points for Music Flamingo and +0.84 for MOSS-Music, against +18.29 and +3.50 for Relation MultiQA alone. The two roles teach different things: holding several inputs apart, and knowing what relates them.

Training on both beats either alone, and by more than the two add up to: +28.24 against 23.84 for Music Flamingo, +6.88 against 4.34 for MOSS-Music. Single-audio accuracy stays above the pre-trained score in every condition.

6. CONCLUSION

STEMMA uses song–section–stem provenance to generate relation-first multi-audio QA and instruction-tuning data. On STEMMA-Bench’s structural relations, both tuned models outperform their corresponding caption cascades, whereas a cascade achieves the highest score on Jamendo-MT-QA. Combining Relation MultiQA with coreference yields the highest multi-audio average for both backbones while keeping their single-audio averages above pre-trained levels.

7. ACKNOWLEDGMENTS

This work was supported by the Korea Creative Content Agency (KOCCA) and the Artificial Intelligence Industry Cluster Agency (AICA).

8. REFERENCES

- [1] Joshua P. Gardner, Simon Durand, Daniel Stoller, and Rachel M. Bittner, “LLark: A multimodal instruction-following language model for music,” in *Proc. ICML*, 2024, pp. 15037–15082.
- [2] Sreyan Ghosh, Arushi Goel, Lasha Koroshinadze, Sang-gil Lee, et al., “Music Flamingo: Scaling music understanding in audio language models,” in *Proc. ICLR*, 2026.
- [3] Mark Mulligan, “Future of the music creator economy: Tipping point,” Tech. Rep., MIDiA Research, June 2026.
- [4] Alain Riou, Joan Serrà, and Yuki Mitsufuji, “Automatic music sample identification with multi-track contrastive learning,” in *Proc. IEEE ICASSP*, 2026, pp. 14587–14591.
- [5] Wonil Kim, Hyeonseok Wi, Seungsoon Park, Taejun Kim, et al., “From generation to attribution: Music AI agent architectures for the post-streaming era,” in *NeurIPS Workshop on AI for Music*, 2025.
- [6] Alain Riou, Antonin Gagneré, Gaëtan Hadjeres, Stefan Latner, and Geoffroy Peeters, “Zero-shot musical stem retrieval with joint-embedding predictive architectures,” in *Proc. IEEE ICASSP*, 2025, pp. 1–5.
- [7] Dingyao Yu, Kaitao Song, Peiling Lu, Tianyu He, et al., “MusicAgent: An AI agent for music understanding and generation with large language models,” in *Proc. EMNLP: System Demonstrations*, 2023, pp. 246–255.
- [8] Yiming Chen, Xianghu Yue, Xiaoxue Gao, Chen Zhang, et al., “Beyond single-audio: Advancing multi-audio processing in audio large language models,” in *Findings of EMNLP*, 2024, pp. 10917–10930.
- [9] Sonal Kumar, Sreyan Ghosh, Yueqian Lin, S. Sakshi, et al., “PolyAudio: Advancing multi-audio reasoning in large audio language models with interleaved multi-audio contexts,” in *Findings of ACL*, 2026, pp. 42335–42353.
- [10] Junyoung Koh, Jaeyun Lee, Soo Yong Kim, Gyu Hyeon Choi, et al., “Jamendo-MT-QA: A benchmark for multi-track comparative music question answering,” in *Findings of ACL*, 2026, pp. 2612–2631.
- [11] Soham Deshmukh, Shuo Han, Rita Singh, and Bhiksha Raj, “ADIFF: Explaining audio difference using natural language,” in *Proc. ICLR*, 2025.
- [12] Chih-Kai Yang, Yun-Shao Tsai, Yu-Kai Guo, Ping-Le Tsai, et al., “MUGEN: Evaluating and improving multi-audio understanding of large audio-language models,” in *Proc. Interspeech*, 2026, pp. 3803–3808.
- [13] OpenMOSS Team, “MOSS-Music,” <https://github.com/OpenMOSS/MOSS-Music>, 2026, GitHub repository.
- [14] Abhinaba Roy, Renhang Liu, Tongyu Lu, and Dorien Herremans, “JamendoMaxCaps: A large scale music-caption dataset with imputed metadata,” in *Proc. IJCNN*, 2025, pp. 1–8.
- [15] Michaël Defferrard, Kirell Benzi, Pierre Vanderghenst, and Xavier Bresson, “FMA: A dataset for music analysis,” in *Proc. ISMIR*, 2017, pp. 316–323.
- [16] Chunbo Hao et al., “SongFormer: Scaling music structure analysis with heterogeneous supervision,” *arXiv preprint arXiv:2510.02797*, 2025.
- [17] Weinan Tong et al., “SCNet: Sparse compression network for music source separation,” in *Proc. IEEE ICASSP*, 2024, pp. 1276–1280.
- [18] Dmitry Bogdanov, Nicolas Wack, Emilia Gómez, Sankalp Gulati, et al., “Essentia: An audio analysis library for music information retrieval,” in *Proc. ISMIR*, 2013, pp. 493–498.
- [19] Pablo Alonso-Jiménez, Xavier Serra, and Dmitry Bogdanov, “Music representation learning based on editorial metadata from Discogs,” in *Proc. ISMIR*, 2022, pp. 825–833.
- [20] Zafar Rafii, Antoine Liutkus, Fabian-Robert Stöter, Stylianos Ioannis Mimilakis, and Rachel Bittner, “MUSDB18 – a corpus for music separation,” Zenodo, doi: 10.5281/zenodo.1117372, 2017.
- [21] Rachel M. Bittner, Justin Salamon, Mike Tierney, Matthias Mauch, Chris Cannam, and Juan Pablo Bello, “MedleyDB: A multitrack dataset for annotation-intensive MIR research,” in *Proc. ISMIR*, 2014, pp. 155–160.
- [22] Igor Pereira, Felipe Araújo, Filip Korzeniowski, and Richard Vogl, “MoisesDB: A dataset for source separation beyond 4-stems,” in *Proc. ISMIR*, 2023, pp. 619–626.
- [23] Sonal Kumar, Šimon Sedláček, Vaibhavi Lokegaonkar, Fernando López, et al., “MMAU-Pro: A challenging and comprehensive benchmark for holistic evaluation of audio general intelligence,” in *Proc. AAAI*, 2026, vol. 40, pp. 22688–22697.
- [24] S. Sakshi, Utkarsh Tyagi, Sonal Kumar, Ashish Seth, et al., “MMAU: A massive multi-task audio understanding and reasoning benchmark,” in *Proc. ICLR*, 2025, pp. 84929–84964.
- [25] Ziyang Ma, Yinghao Ma, Yanqiao Zhu, Chen Yang, et al., “MMAR: A challenging benchmark for deep reasoning in speech, audio, music, and their mix,” in *Proc. NeurIPS*, 2025, vol. 38, pp. 74837–74868.
- [26] Yongyi Zang, Sean O’Brien, Taylor Berg-Kirkpatrick, Julian McAuley, and Zachary Novack, “Are you really listening? Boosting perceptual awareness in music-QA benchmarks,” in *Proc. ISMIR*, 2025, pp. 247–261.
- [27] Qwen Team, “Qwen3.6-35B-A3B: Agentic coding power, now open to all,” <https://qwen.ai/blog?id=qwen3.6-35b-a3b>, April 2026.
- [28] Sreyan Ghosh, Arushi Goel, Jaehyeon Kim, Sonal Kumar, et al., “Audio Flamingo 3: Advancing audio intelligence with fully open large audio language models,” in *Proc. NeurIPS*, 2025, vol. 38.
- [29] Chen Yang, Chufan Yu, Hanfu Chen, Jie Zhu, et al., “MOSS-Audio technical report,” *arXiv preprint arXiv:2606.01802*, 2026.
- [30] Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, et al., “Qwen2-Audio technical report,” *arXiv preprint arXiv:2407.10759*, 2024.
- [31] Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, et al., “Qwen2.5-Omni technical report,” *arXiv preprint arXiv:2503.20215*, 2025.